Programming Language Concepts

Notes by Arjun Maneesh Agarwal

based on course by Madhavan Mukund and SP Suresh

Table of Contents

1. What is this course about?

There is a whole zoo of languages: Haskell, Python, Java, JS, Swift, Go, Lean, Rust, C, C++ ect. A question worth asking is 'Why so many?'

There are many human languages due to geography and culture but programmer's all want to do the same thing. Are these languages different? If yes, how?

Let's start with something concrete: Haskell is functional and Python is imperative. But are the definitions clear? Intuitively, sure but. trying to do so formally is not possible. For example: Haskell has imperative constructs like monads and IO while Python has function features like map, filter and list comprehension.

A slightly better description for Haskell would be declarative. This is as we don't usually give step by step instructions but declare what we want and the compiler decides the ordering and sequence. While in an imperative language, we tell the exact recipe hoping that the cake that comes out is good (for example the procedure of carry addition doesn't trivially mean that addition will take place).

As they both exist, that must imply that we might be better off using declarative and imperative at different times and tasks. For example: Declarative languages abstract out the management of resources (like memory) while Imperative Languages allow a finer control over resources; but proving the correctness of a programme is much easier in a Declarative programme than in a Imperative as there there is a disconnect between "intent" and "instructions".

1.1. Language Complexity

Let's consider a function (which will exist in both the languages) which just takes in parameters and returns the result.

Consider sorting the population of India in say Haskell. We really don't want to make a copy of the list as that would waste a lot of space. What we really want is sorting but with inplace updates (which would need us to manage how the data is stored in the memory).

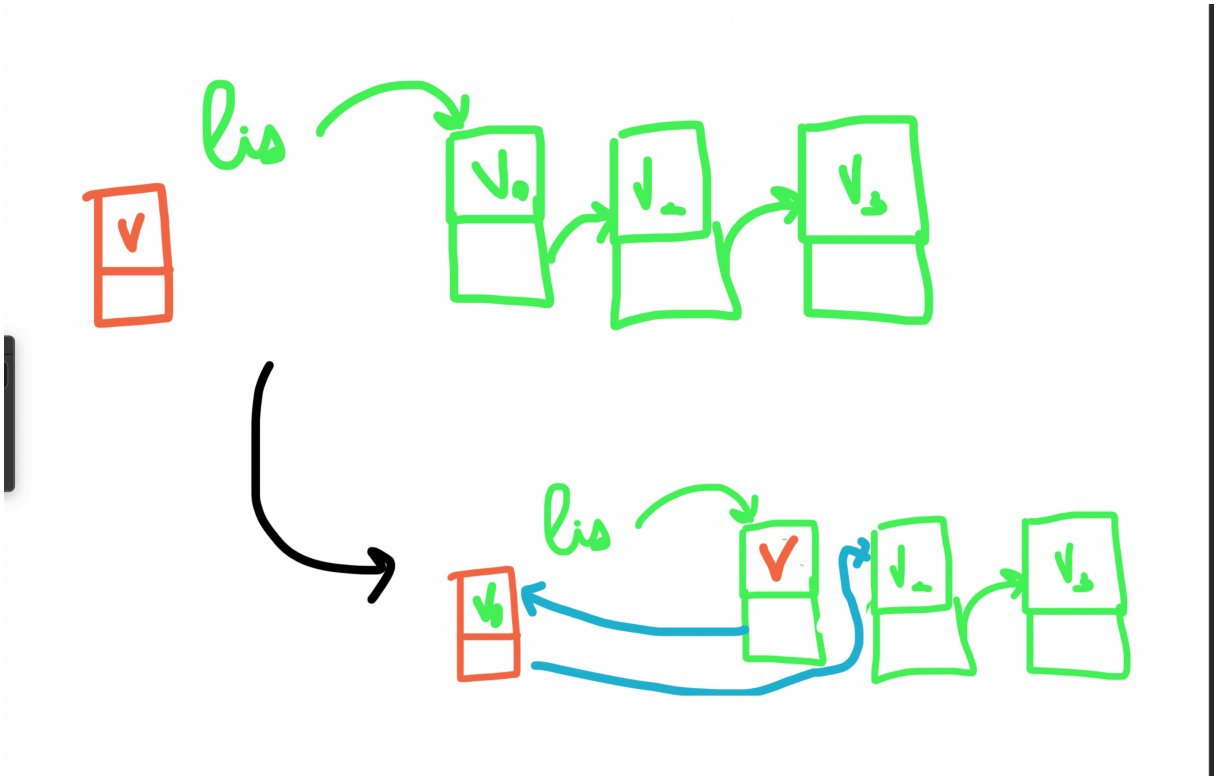
Consider the language Pascal[1]:

```
def f(x, var y):
  <code>
```

here var tells the language where `y` can be updated inplace while we make copies of say `x`. However, many of languages (none of the ones we mentioned) don't have such a choice as it is yet another thing for the programmer to keep track of. In general, a choice has to be made at time of making the language that either no argument can be updated or any argument is updatable.

But clearly the former is less dangerous as we don't know where else the argument could be passed. This leads to the idea of `&` in C++ and others where instead of passing the variable we pass the address and ask the function to look at whatever is stored at the address.

Although, we don't have this in Python. Python arbitrarily decides if the data structure is mutable or immutable. For example, lists and dictionaries in Python are implicitly passed by the address. This is called 'pass by value' and 'pass by reference' where the former makes a copy while the latter doesn't. Technically, by convention, we always pass by value. The value is just sometimes a location. Although, we can never modify the location address. Here is an example of how `insert(l,v)` would work in Python.



2. What is datatype?

[1] which was an old language which is now rarely used